Project Title:

done!

Student No:

20108892

Student Name:

Noemi Lovei

Table of Contents

Project Title: 1

done! 1

Student No: 1

Student Name: 1

Table of Figures 3

Table of Tables 4

Declaration of Authenticity 5

Executive Summary: 5

Background: 5

Introduction: 6

RAMP: 6

Research & Analysis 6

Requirements Analysis: 7

Functional Requirements: 7

Non Functional Requirements: 9

Feature Analysis: 9

Market Analysis: 10

Popular Features: 11

Current Technologies: 11

Colour Schemes: 12

Repo Analysis: 13

Methodology Statement: 13

Modeling: 14

Jane's Story: 15

Bob's Story (Admin) 15

Database Description: 15

Use of AI in project: 16

Planning: 17

Iteration Plans 17

Tabular Planning: 18

Sprints: 18

Execution: 21

3rd Party Helper Packages 21

Implementation details 22

What you have achieved 22

Future Features 25

Problems Encountered 25

What you have learned 26

Appendix: 27

Bibliography: 41

Table of Figures

Figure 1 27
Figure 2 27
Figure 3 28
Figure 4 28
Figure 5 28
Figure 6 29
Figure 7 29
Figure 8 29
Figure 9 30
Figure 10 30
Figure 11 31
Figure 12 31
Figure 13 32
Figure 14 32
Figure 15 32
Figure 16 33
Figure 17 33
Figure 18 33
Figure 19 34
Figure 20 34
Figure 21 35
Figure 22 35
Figure 23 36
Figure 24 36
Figure 25 37
Figure 26 37
Figure 27 38
Figure 28 38
Figure 29 39
Figure 30 39
Figure 31 40
Figure 32 40

Table of Tables

Table 1 6
Table 2 7
Table 3 9
Table 4 10
Table 5 11
Table 6 11
Table 7 12
Table 8 13
Table 9 18
Table 10 18
Table 11 19
Table 12 19
Table 13 19
Table 14 20
Table 15 20
Table 16 20

Declaration of Authenticity

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student: *Noemi Lovei* Date: *28th March 2026*

Executive Summary:

A gamified tracker of all things done available on a webapp, supplemented with in-game currency and in-app statistics and charts for visualising user engagement. Built with a nodejs backend exposing the backend APIs, storing data through a MYSQL database and a react frontend all the while connected to a custom python mini AI server making recommendations based on user activity and delivering through web API. Built with robust front-end design, keeping data security and good database practices in mind.

Background:

The problem statement: People focus on the todos so much that they don't get to celebrate the things that they have been done. The purpose of the project is to allow for people to track their achievements (done tasks), celebrate along the way and ultimately reflect and refine their habits.

Unlike a traditional todo list which is a list of tasks crossed off and never again looked at, here I am building a prototype that is meant to be looked after a task is completed, meant to notice any patterns and make decisions to re orientate and realign to personal goals, by reflecting on existing habits and re-evalauting those.

The application will further leverage gamification as a tool to engage the user and turn them in to regular users. Why? A recent study in UCD highlighted that gamification is effective for creating increased awareness and broader engagement (Macori et al., 2025:12).

Introduction:

I am creating a prototype that will allow users to add their achievements, per different categories and then reflect on these achievements using different visuals, charts and filtering.

I am proposing to use the following technologies, nodejs & hapi for backend, react for front end and javascript as the main programming language with data storage being handled by mongodb.

I am proposing to build a web app instead of a mobile app to leverage a wider accessibility & cross platform compatibility. It also requires lower development time and cost as I am building only one webapp and not platform specifc apps which is the perfect method for a prototype. Further for demonstration purposes no installation is required to explore the features of this prototype.

The benefit from the dopamine rush of receiving rewards and encouragement well documented to influence human behaviour, "is linked to alterations in the dopaminergic reward system" (Weinstein and Lejoyeux 2015: 120) and I will leverage gamification in the prototype to encouarge positive engagement with the app.

RAMP:

Research & Analysis

Table 1

Front End Requirements:	**Back End Requirements:**
Must offer flexibility for growth	Must be I/O Intensive
Be able to handle lots of data	Have a good eco-system
Must be future proof	Adhere to general security & compliance standards
Must be fast & consistent	

I propose to use React as the main front end library for the prototype because it meet's all the criteria outlined. It offers flexibility in choosing my own stack and works well with many styling libraries which will be important for the gamification and UI of the prototype. Through logic reuse it is quite fast and consistent and will be able to handle lots of data coming from the backend and potential AI integration.

For the backend I propose to use Hapi.js because it is an open-source and enterpise grade Node.js framework. Hapi.js is known for it's security and predictability which are a must for this prototype. Choosing stable technology ensures that the prototype can be developed further and won't face a situation where the technology becomes irrelevant.

Requirements Analysis:

Functional Requirements:

Table 2

As a daily user who wants to check-in my activities, I want to make a log quickly.	Acceptance Criteria: - The user sees the check-in form and the submit button - Can easily select which category they are adding an activity to - The user can add optional description - Clicking Save records the activity and redirects the user to add more activities
As a daily user I want to be able to set my own categories on my activities	Acceptance Criteria: - The user can add own options of categorisation - The user added categorisation will appear along side the pre defined categorisations
As a daily user I want to make logs easily.	Acceptance Criteria: - The user is brought to login page as the first thing in the app - The user doesn't have to go looking for the check-in option - The user can add new days to add their achievements to
As a daily user I want to reflect on the activities that I have logged:	Acceptance criteria: - The user can easily find the reflect view - The user can filter through days and categories of activities - The user can view different kinds of charts
As the user I want to be able to provide feedback to the app.	Acceptance criteria: - The user sees the feedback form and submits - The user feedback appears on the screen like in mail style - The user recives a reply from the admin
As the user I want to receive AI suggestions to help my reflection:	Acceptance criteria: - An ML model is implemented successfully - Suggestions are made to the user - The suggestions are relevant

As the user I want to be rewarded for actions logged.	Acceptance criteria: - Popup when user logs an event - Credit is issued to the user upon event log
As the user I want to be able to spend my rewards.	Acceptance criteria: - The shop view exits with buttons to buy - The user can select smth to buy and this reduces their credit - The purchased items appear in their private gallery.
As the user I want to share my rewards with social media.	Acceptance criteria: - Share button integration from private gallery to 3rd party socials
As the admin user, I want to be able to see all users and perform RD on them.	Acceptance criteria: - Admin account exists - Admin can log in and has admin only views - Can see all users - Can filter among users - Can delete users (or archive accounts)
As the admin user I want to be able to send custom replies to the user complaint.	Acceptance criteria: - The admin can go to the mail view and reply to the comment made by the user

Non Functional Requirements:

Table 3

Authentication	Acceptance criteria: - The users must log in to their individual accounts - Must use strong password hashing
Authorization	Acceptance criteria: - Admins can log in to admin accounts
Vulnerability Protection	Acceptance criteria: - Must have form validation to prevent sql injection attacks
Usability	Acceptance criteria: - There must be UI Consistency - New users must be able to complete simple tasks with minimal instruction
Colour Scheme	Acceptance criteria: - Must match the gamification spirirt

Feature Analysis:

My users should be able to:

- Login
- Add new days
- Add new achievements
- Add custom categories
- Filter achievements per day and see their categories
- Earn rewards
- Spend rewards in shop
- Display purchases in private gallery
- Share purchases to social media
- View their achievements in a tree visual
- View achievements in different charts (pie, bar)
- Provide feedback to the admin

My admin users should be able to:

- View all users
- Take actions on user accounts
- React to feedback provided by users
- Needs to be able to update the coins balance for a user

Market Analysis:

I am going ot explore the market in 3 different categories, To Do Apps, Habit Trackers and Reflection Apps in this table. I am examining the different features that these varied existing apps have.

Table 4

To Do Apps		Habit Trackers		Reflection Apps	
Todoist =>	Quick add tasks Send reminders Recurring dates Filter todo's by category Has flexible views Collaboration features Has project templates	Way of Life = >	Simple and fast Track anything Dark mode Reminders Charts	Reflectly =>	Personal journaling AI questions Tracking Personalized content
TickTick =>	Add tasks List, filter, tag Reminders Calendar view Eisenhower matrix Collaboration Keyboard shortcuts	Habitify =>	Planning Visualise streak Sync with Google Fit Mood tracking Reminders	Finch =>	Mood tracking Mindful goals and exercises Companion growing animation Collect micro-pets
Microsoft To Do =>	Daily planner Lists Tasks and subtasks Importance ranking Notes & attachments Reminders	Habitica =>	Get rewards To Do's Habits Team up with friends Battle monsters Challenge each other		
Toodledo =>	Organize tasks Productivity Tools Collaboration Data Management	Notion =>	Product Roadmap 3rd party importing AI features		
Sunsama =>	Calendar view Mute other apps Visualisations Integrate 3rd party apps				

Popular Features:

Examining the most popular features that exist among the market and competitors.

Table 5

	Add tasks	List tasks	Filter tasks	Reminders	Diff views	Visuals	Collab	Rewards	3rd party integration
Todoist	X		X	X	X		X		
TickTick	X	X	X	X	X		X		
Microsoft To Do	X	X		X					
Toodledo	X		X				X		
Sunsama	X				X	X			X
Way of Life	X				X				
Habitify	X			X		X			X
Habitica	X						X	X	
Notion	X				X		X		X
Reflectly	X				X		X		
Finch	X				X			X	

Analysis:

The prominent features among the competitors are the ability for the user to add their tasks/habits/goals in to the UI and view their achievements in different ways. Interesting features that I would like to explore further are the collab features (integration with social media) and for my gamification goal visual features are a must.

Current Technologies:

Examining some of the technology likely used by competitors. As not all of their stack is available to the public, the table is an estimate and not an accurate literal statement.

Table 6

	Web/Mobile	Sentry	Firebase	SDKs	Javascript	C#	React	SQL	Node js
Todoist	Mobile	X	X	X					
TickTick	Mobile	X	X	X					
Microsoft To Do	Both				X	X			
Toodledo	Mobile				X		X		
Sunsama	Both						X	X	X
Notion	Both						X		
Way of Life	Mobile		X						
Habitify	Both		X	X			X		
Habitica	Realm		X						
Reflectly	Both						X		

Analysis:

React is a popular choice for front end both mobile and web apps. I also note that Firebase is a very good choice for backend due to the varied features that it provides such as out of box security and potential integration with its own db.

Colour Schemes:

Table 7

	Most Predominant	Most Predominant	Overall
Todoist	White	Orange	Pastel
TickTick	White	Blue / Green	Blue Scheme
Microsoft To Do	White	Blue	Microsoft
Toodledo	White	Blue	Blue Scheme
Sunsama	White	Orange	Orange Scheme
Notion	White	Yellow	Spring Colours
Way of Life	Green	White	Green Scheme
Habitify	White	Blue	Blue Scheme
Habitica	Purple	White	Rich Colours
Reflectly	Violet	White	Violet Scheme
Finch	Blue	Green	Spring Colours

Analysis:

It's not surprising that apps aimed at working professionals in the context of goal setting mostly choose soft and neutral colours such as white. A recent study confirmed that colours trigger different emotions in people Jonauskaite and Mohr (2025). In line with Habitify I am going to go with a stronger colour scheme to reflect the more gamified element of this prototype.

Repo Analysis:

Table 8

Git Hub Repo	Compare & Contrast
LifeUp: https://github.com/Ayagikei/LifeUp	1. Similar in concept to my proposed prototype 2. Mobile app where as I m building a desktop version. 3. Highly gamified to promote engagement
Quest Tracker: https://github.com/atayl16/quest_tracker	1. Heavy RPG style gamification which appeals to a nicher audience 2. Built with Ruby on Rails
Gamified Habit Tracker https://github.com/BetterACS/GHT	1. Roleplay gamification 2. Uses images as “rewards”

Methodology Statement:

The Agile methodology will be used to develop this prototype. Work will be broken down in to smaller sprints and each sprint will deliver an increment to the product. I will use the Kanban framework and Trello to plan the sprints.

Modeling:

Jane is a typical user, born in Ireland, she is a hard worker, completed 3rd level education and is currently pursuing a career in Law. With constant pressure to meet deadlines and never ending to do lists, she has decided to try something else and start tracking all the achievements she has completed and reflect on those to see if she can identify any patterns, opportunities for improvement in her daily habits and to practice mindfullness and awarness on her own hard-work. She knows from her studies that before jumping in to a solution, she first needs to observe how her achievements look like and then make data driven decisions from there. She doesn't have much free time, but she likes to go running to clear her mind and keep fit. She dislikes wasting time and ineffective processes.

Joe is a typical user, born in Ireland, he has finished secondary school and an apprenticeship and is a qualified electrician. He likes his work and is passionate about making money. He dislikes rash decisions and is always prudent in his work and life. He has decided he wants to be his own boss and start as a sole trader, but before he can do that he needs to assess his current achievements and personal habits to decide whether he is ready for this big step now, or if he first needs to make some changes to be in the best possible position for a successful startup.

Bob works in IT and he was hired to act as admin user of the Done! App. His job is to interact with users and their queries and manage user accounts. He likes analysing data and observing user behaviour and dislikes chaos.

Jane's Story:

Jane as a typical user needs to login to the webapp so we need a login view. See Figure 1. If this is the first time Jane is using the app she will need to sign up so we need a sign up view. See Figure 2. All of the information from the form needs to be stored in our DB. See Figure 3. When Jane logs in successfully, she will land in the dashboard view, where she can add new days. See Figure 4. Jane will see her days listed on the same dashboard view. Jane may want to delete an accidently added day and she may want to archive a day to reduce the clutter in the dashboard. See Figure 5. She can find the archived days in the archived view. They can be restored to the calendar view if needed. See Figure 6. All the day related activities need to be reflected in the DB, along with the actual datestamp for filtering purposes in the future. See Figure 7. Jane will then want to add achievements in to her days so we need a day view. The categorisation is hard coded in to the prototype. See Figure 8. The achivements will be seen on the same day view. See Figure 9. Jane may decided that she wants to add her own categorisation of her achievements. See Figure 10. The new UI with the new category would look like this with a scrollbar. See Figure 11. She may realise that she added the wrong details and she will want to edit the achivement or delete it all together. See Figure 12. When Jane adds an achievement this should show on her stats score. See Figure 13. When she clicks edit, an edit view form will be triggered. See Figure 14. Coins will be calculated for her after adding an achievement and/or updating. See Figure 15. For each achievement 10 coins will be added to her purse. See Figure 16. For more advanced calculation, the weight of the achievement should determine how much coins are added. See Figure 17. Curious to spend her coins Jane will visit the shop view. See Figure 18. She will then want to see her purchases, by visitng the gallery view. She can also share her achievements to social media. See Figure 19. Jane can then visit the Reflection view where she can filter her tasks. The Bar and Pie Charts will appear on this page based on the filters. See Figure 20. Jane can view her tree of progress which is a visual representation. See Figure 21. AI can make suggestions to the user based on their past achievements. See Figure 22. Jane can click on the AI button and a personalised suggestion will appear for her. See Figure 23. She may want to edit her account details which she can do from the accoints view. See Figure 24. She may want to send some feedback about the product by contacting admin. See Figure 25. This she can do on the ContactAdmin View. See Figure 26. The admin will receive and reply to Jane's message. See Figure 27.

Bob's Story (Admin)

The admin can login and see a list of all users; The admin can delete a user by clicking on the red button. See Figure 28. Bob can view messages from users and reply. See Figure 29. The Admin can send coins to the user manually be it as a reward or compensation with regards to a complaint. The Admin user can also upload items to the shop and set the price. See Figure 30.

Database Description:

As per Figure 32 there are 5 tables inside the database. The Users table, the Tasks table, the Dates table, the shopItems table, the Messages table.

Use of AI in project:

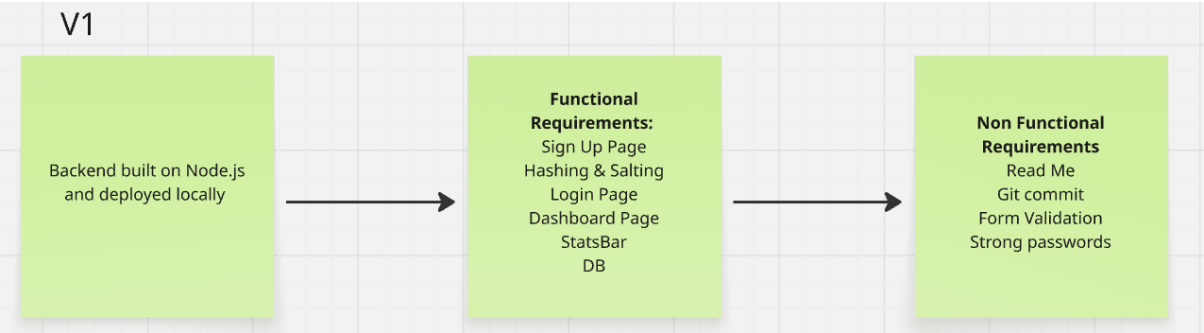
Generated images on Gemini 3 for the shopPage on 22.12.2025

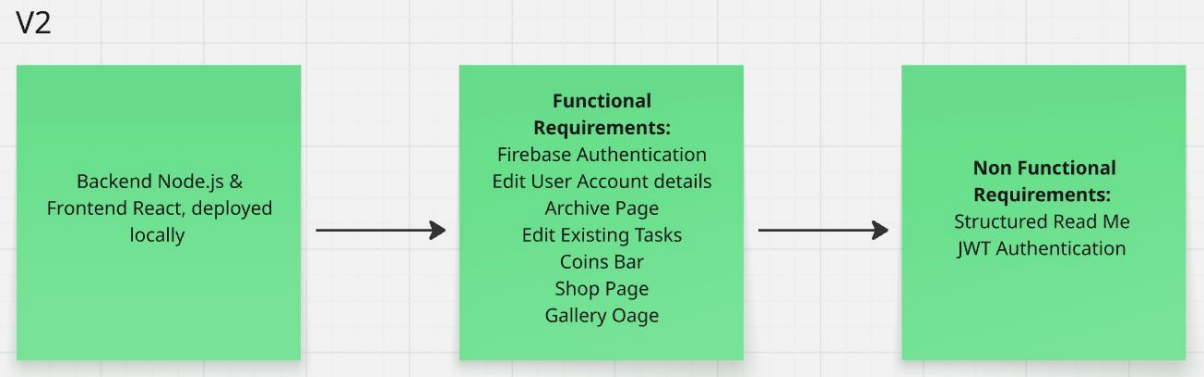
(Sample)

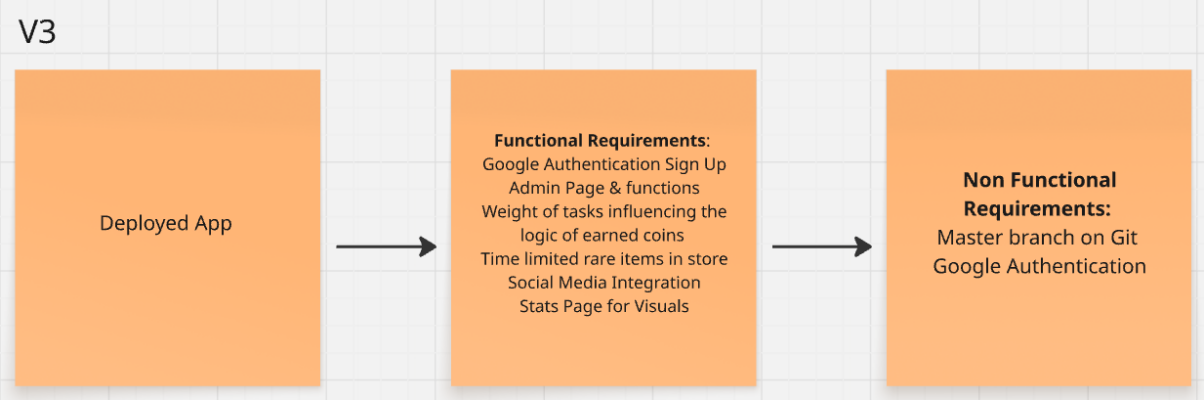
Generated logo image on Gemini 3 for the LoginPage on 08.02.2025

Planning:

Iteration Plans







Tabular Planning:

12 weeks, 8 hours each = 96 hours in total or 6 sprints 16 hours each.

Table 9

V1	V2	V3	V4
ReadMe			
SignUp Page & hashing & salting	Firebase Authentication	Goole Authentication Sign up	AI Integration per user
Login Page	Edit User account details	Admin Page & functions	
Days Dashboard Page	Archive Page for days	Forgotten password?	
Tasks Page	Edit existing tasks	Weight of tasks influencing the logic of coins earned	User can add their own task classifications
StatsBar	Coins Bar		
	Shop Page	Time limited rare items in store	
	Gallery Page	Social media integration for sharing purchases	
	Contact Admin feature	Stats Page with visuals for reflection	Tree of progress – animated visuals
		Deploy prototype	

Sprints:

Sprint 1

Table 10

Description	*Time Required*	
Ensure MongoDb connection is live with backend	1 hour	Completed
Move the db to MongoAtlas	2 hours	Completed
Ensure backend contains the following features: Date Obj Task Obj LoginPage SignUp Page Dashboard TaskView Page	8 hours	Completed
Ensure APIs are working through Postman	1 hour	Completed

Sprint 2

Table 11

Description	*Time Required*	
Ensure FrontEnd has the following features: LoginPage SignUp Page Dashboard TaskviewPage	4 hours	Completed
Ensure Firebase Authentication is working	1 hour	Completed
Set up CoinsBar & Stats Bar in the backend with appropriate APIs exposing it to the front end	4 hour	Completed
Ensure CoinsBar & StatsBar are available on the frontEnd	4 hour	Completed

Sprint 3

Table 12

Description	*Time Required*	
Ensure shop logic and gallery logic are built in the backend	5 hours	Completed
Use Firebase Storage combined with mongo db storage for the images in the shop	2 hours	Completed
Build Shimmer UI for Shop	0.5 hours	Completed
Ensure ShopPage and Gallery page are built in the frontend	4 hours	Completed
Ensure the Archive page is fully functional for both backend and front end	4 hours	Completed

Sprint 4

Table 13

Description	*Time Required*	
Ensure ReflectionPage is built with Visual	4 hours	Completed
Create different types of visuals?	4 hours	Completed
Add Admin User through non exposed API	2 hours	Completed

Sprint 5

Table 14

Description	*Time Required*	
Merge the two github projects in to one. Write detailed ReadMe for prototype usage	2 hours	Completed

Sprint 6

Table 15

Description	*Time Required*	
AI Integration	16 hours	Completed

Trello Sprints

Table 16

Sprint One	https://trello.com/b/qKa97gmJ/sprint-1
Sprint Two	https://trello.com/b/3q1LXKff/sprint-two
Sprint Three	https://trello.com/b/gi7WT1dw/sprint-3
Sprint Four	https://trello.com/b/jUWQcRi8/sprint-4
Sprint Five	https://trello.com/b/R2FMlykv/sprint-5

Execution:

3rd Party Helper Packages

Name:	*Reason:*
Fast API	For AI server to handle web requests
Pandas	Data processing
NumPy	Data processing
React-dropzone	Drag and drop file uploads
Framer Motion library	Create animations
Lucide React	Icons
React-toastify	Notifications
React E Charts	Visuals
Bulma	CSS

Implementation details

What you have achieved

The app is available on: https://done-6b1f3.web.app/login

I have implemented a backend using node js. I have 5 API files that interact with their respective 5 DB files. (date-mongo-store, task-mongo-store, user-mongo-store, message-mongo-store, shop-mongo-store) The APIs available are as follows:

date-api.js

1. getAllDates
2. getDateById
3. addDate
4. getUserDates
5. getUserDatesArchived
6. getUserDatesNotArchived
7. deleteDateById
8. deleteAllDates
9. toggleArchiveStatus
10. filterByRange

task-api.js

1. getAllTasks
2. addTask
3. getTasksByDateId
4. getTasksByUserId
5. getTasksByClassification
6. getTaskListByClassification
7. getWeightByClassification
8. getTaskById
9. deleteTaskById
10. deleteTaskByDateId
11. updateTask

user-api.js

1. authenticate
2. getAllUsers
3. getUserById
4. updateUser
5. addUser
6. updateBalance
7. deleteUserById
8. deleteAll
9. deleteDateCascadeTasks
10. buyItem
11. getInventory

shop-api.js

1. getShopItems
2. addStock
3. sharePreview
4. addImage

message-api.js

1. getAllMessages
2. addMessage
3. getConversation
4. getAdminInbox

Of the 40 possible API endpoints only 38 API endpoints are exposed by the backend for the front end to avail of.

I have built the front end using the react javascript library. I have 27 components, 4 navbar components , 2 context components, 6 custom hooks and numerous other helper components.

The Login component is the logical start of the webapp and therefore the root url. From there the ExternalNav allows navigation to the Sign up component if required. I also have a forgotten password link embedded. The next component is the Dashboard component which is itself made up of the DayForm component and ListDays components and an Internal Nav. This allows the props to be seemlessly passed among the DayForm and ListDays components which allows for any new days added to appear immediately in the list. Further more I have built the custom hooks useDayList and useDayLogSubmission to extract some of the code from the components making the readability of the logic easier.

After building the Dashboard component I have added the TaskPage component. To this I have added the ListTask and TaskForm helper components. I originally didn't build a shimmer UI but after discovering how slow the data loads from the database I made sure to add a filter for a better experience for the user. This is the first component where we see the SubNav. I had explored different ways of placing the Coins component and Stats component across different views, but in the end decided on a SubNav to unify the UI between this TaskPage component and the upcoming Shop Component. The TaskPage further contains the first modal component for the purpose of editing any tasks after they have been added. I have used modal style components throughout the web app after discovering how useful they are while building the TaskPage.

I have built an ArchivePage component so that the user can remove all the old dates from their view. The user can easily restore such dates back to the list of dates, should they change their minds or wish to view the details once again.

For gamification purposes I built a Shop component where the user can purchase virtual goods using the coins they have earned on the TaskPage. The Shop component contains a variation of the SubNav whereby the coins are visible to the user, but their stats are not. I explored using first the built in alert pop-up and then a toast pop-up to inform the user of a successful purchase, but then decided on a custom Success component that better suits the colour scheme and mood of the web app. I took inspiration from Duolingo's clean alerts and remove the toast and builtin pop-up. I later decided to reuse this Success component also when a purchase fails and on other views as well.

The next component I built, was very similar to the Shop component but serves the purpose of displaying or collecting the purchases made by the user. While playing with terminology, I have sometimes refered to it as inventory, sometimes as gallery but decided on simply Purchases in the end. The component is called InventoryPage and it makes an API call to the backend to get the user's inventory and displays it inside custom cards, that include a Share Link which satisfies my original goal of allowing the user to integrate their purchases with external 3rd party social media applications. The share link is a quick and easy way to give the user the most flexibility as to which external social media they would like to share their purchases / achievements with. The Inventory Page is not the only view where the user can deliberate on their to-date achievements.

I have built the Reflection component with the aim of displaying a range of different charts to allow the user to see their achievements from different perspectives. This page includes a bar chart and a pie chart for now. The user can also filter per a single day and as suggested by my mentor, a range of days. This allows them to deliberate upon which of their activities requires more attention than others.

I had hoped to build an animated tree that would grow as the user input more and more tasks. I have however parked that idea for the moment. The original plan was the include the AI API calls in to the growing tree, but I have in the end placed the AI API calls in to the Reflection Page. The AI server has been built with Python as opposed to the rest of the web app which is predominantly Javascript. For this reason I best thought to keep the AI server separate from the backend server. As per Appendix 32, I ended up with 3 servers, backend, frontend and AI that I needed to deploy and link together. I considered different technologies such as Amazon AWS for deploying and found it by far the most daunting part of the done! demo application. In the end since I was already using Firebase I decided to deploy the 3 servers with Firebase which allowed for easy management from the Firebase control overall aspects.

I came across Firebase when I was searching for a suitable helper with building authentication security for my app. After much research I decided that Firebase met all the requirements as outlined in table 3 of this report. I chose to use a hybrid solution between mongo atlas and Firebase as illustrated in Figure 31. I made this decision because in the future, should I wish to migrate this project to a different environment, having the DB external will make this transition easier as it can be simply plugged in to the new environment and so I am not overly tied in to Firebase. After working on the authentication with Firebase, I also used it to store all the images required for the Shop component and InventoryPage component as previously discussed. These pages display a large number of images, that I have generated on gemini using prompts inspired by Candy Crush themes. I chose to generate AI images because after research I have found that there are no copyright issues and generally images created using gemini are owned by the creator, this was a crucial factor in deciding to include AI images in the front end.

On the details of the AI server, I used Python instead of Javascript because Python is better suited for data analytics. I availed of NumPy and Pandas libraries both technologies which I have not used prior to this demo project. These libraries perform the analysing and preparing of the user data that is coming from the mongo DB and feed it to the AI model. While I was originally going for AI Clustering, as I don't have enough users in this sample demo, I have instead built a Rule Based Classification, also known as Heuristic Model. The AI server classifies the users in to different categories of achievers and makes suggestions to each user based on their user behaviour. I used FastApi to link the AI server to the frontend. Originally I had very few suggestions available in the AI suggestion bank but after recommendation from my mentor I increased the variety of suggestions in order to reduce the repetition that the user will experience.

The next major component in the frontend is the Profile component where the user is brought to a account form, where they can edit their personal details. They can also reach out to the web administrator from here with any complaints or suggestions that they may have.

I built the Admin UI after I have built the user UI. The Login component is the same, but the user has to have an admin account which is a boolean in the DB and when the admin logs in their view is different. The admin user must be able to delete and communicate with existing users. For this purpose they can see a list of all users on their Admin Dashboard component. I also built a custom error message should someone try to manipulate the URL. There are strict rules forcing the user to be logged in as an admin before the admin dashboard appears. Upon advice from my mentor I added a special ability for the admin to be able to add new stock to the Shop component. This included me building a form where the intended item's image can be uploaded along with the intended price of the said product and once uploaded it goes directly in to the Shop component. In the event of any disputes around coins and coin deletion, I have given the ability to the admin to arbitrarily issue any number of coins to a user. I have built a warning to the admin, that this is intended for exceptional circumstances.

Future Features

For the future of the prototype, I would like to include the originally planned tree animation, that would visually grow as the user adds more and more data. I think that this would be an excellent addition in line with gamification.

Further if the user could select their own classification instead of the currently hard coded classifications, then this would expand the range of organising that the user can benefit from. However for this feature, some re thinking would be required of the existing backend logic to allow for the custom classifications.

Problems Encountered

As I have not initially built the website to be responsive, I had to later tackle the issue with media breakpoints and Bulma formatting. Had I kept responsiveness in the forefront, I would have not run in to design issues where I had to calibrate existing features. For example the subnav-bar with my stats and coins component was working perfectly on the monitor screen, but when trying to scale it down to a smaller screen, such as a phone, the navbar tags couldn't work nicely together with the main navbar. I ended up refactoring the code substantially, wasting time in getting it to work.

I ran into considerable difficulties with getting the correct mongo atlas link to connect to my firebase deployment. The main takeaway was that my username and password login on to the atlas platform and the username and password for the database I was connecting two where two separate identities.

I accidently built a front-end like UI from the backend which I later realised was unnecessary time spent. I only needed the APIs exposed from the backend and didn't need the visual guide. I later ran into some time constraints because of this wasted time and naturally the UI had to be adjusted for the react front end, as designed in the propsal.

A number of the backend API endpoints had to be adjusted when I introduced Firebase to the project. This caused some problems and the logic had to be adjusted, to ensure that all logins are going through Firebase for authentication but that the user data is also stored in my custom db.

What you have learned

I learned the importance of research and analysis before starting a project. From having worked together with my project supervisor, I have found that having someone regularly review the project is very beneficial in helping to identify blindspots.

I learned how to think about what things would draw the customer back to the app. I learnt about thinking of the app as a business, which I had not done before, and thought about monetisation and possible advertisements for the future version. I learnt about the importance of choosing a stable framework that will support my app for a long time and not go out of fashion. This support would mean that there are always engineers and a community available should I need external help with the app.

I learnt about new areas such as web-app accessibility, aria labels and the importance of building an accessible website from the start, rather than making it an after thought.

I learnt to build modals for custom pop ups to allow the user to accept/reject certain actions. I had originally designed the modal as a once off “Success” alert and I kept the name for the component though later I reused this component to mean “Fail” or indeed added extra buttons to allow the user to choose an action. Had I used a more generic approach I wouldn’t have confusing naming in the code.

Appendix:

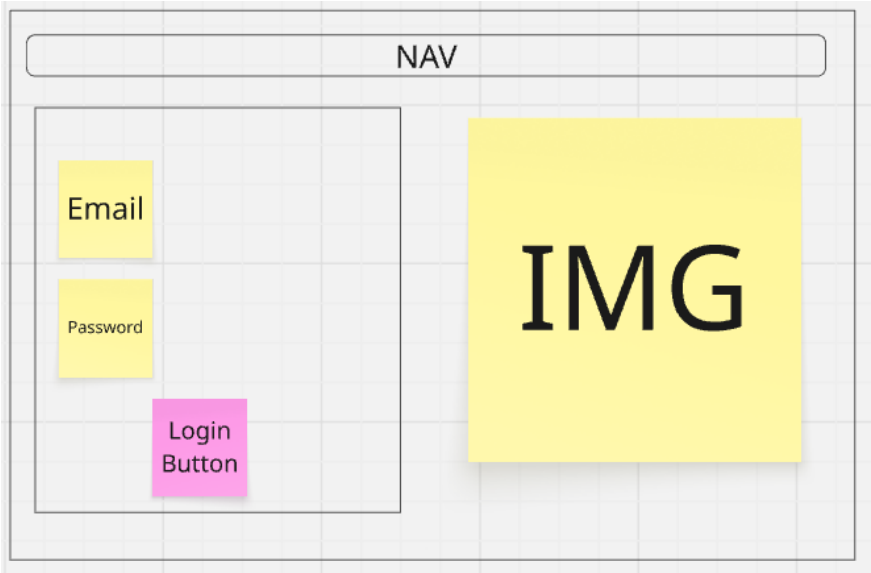


Figure 1

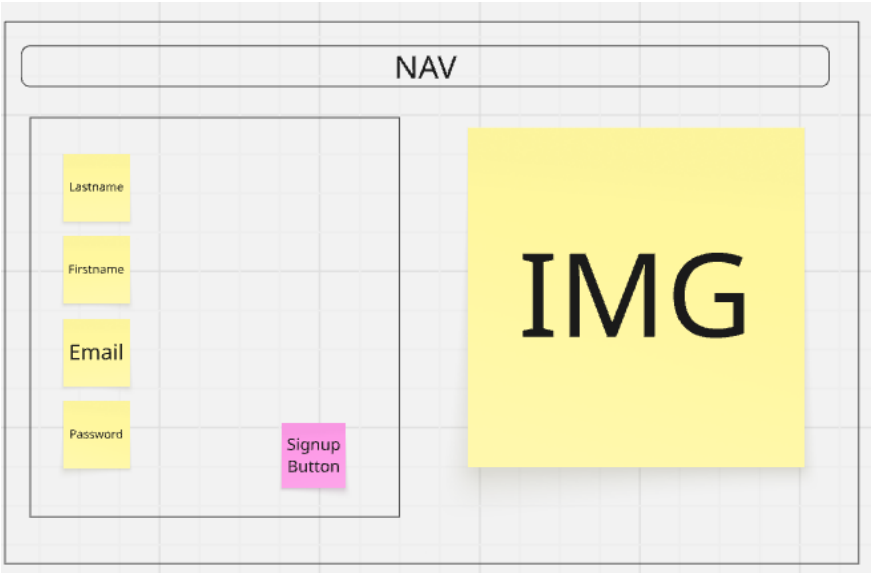


Figure 2

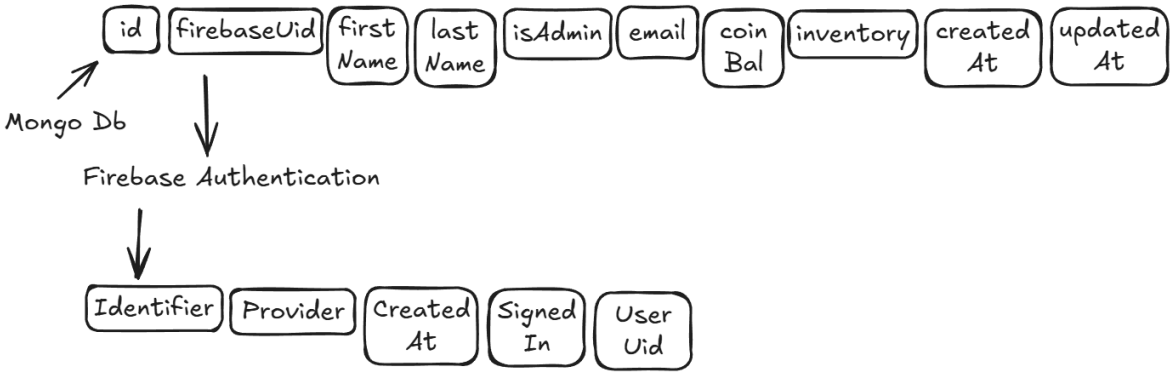


Figure 3

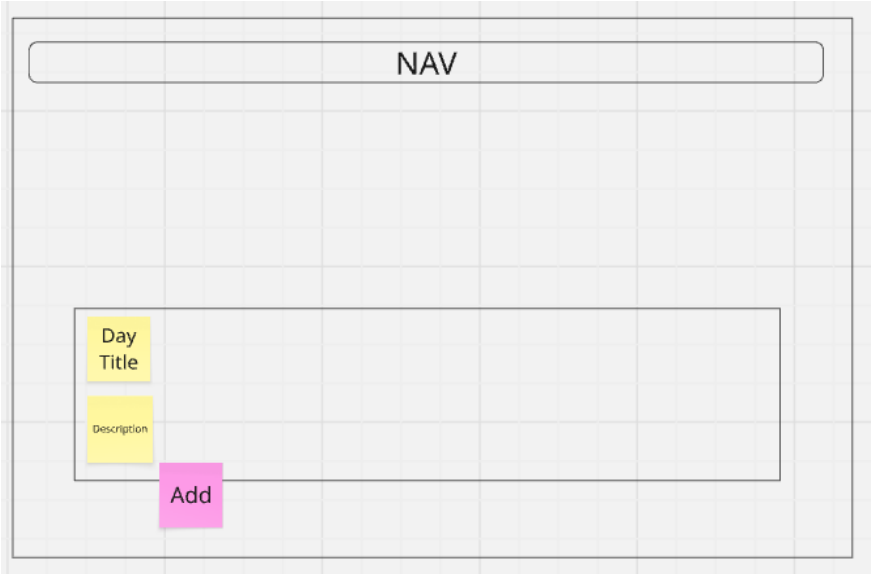


Figure 4

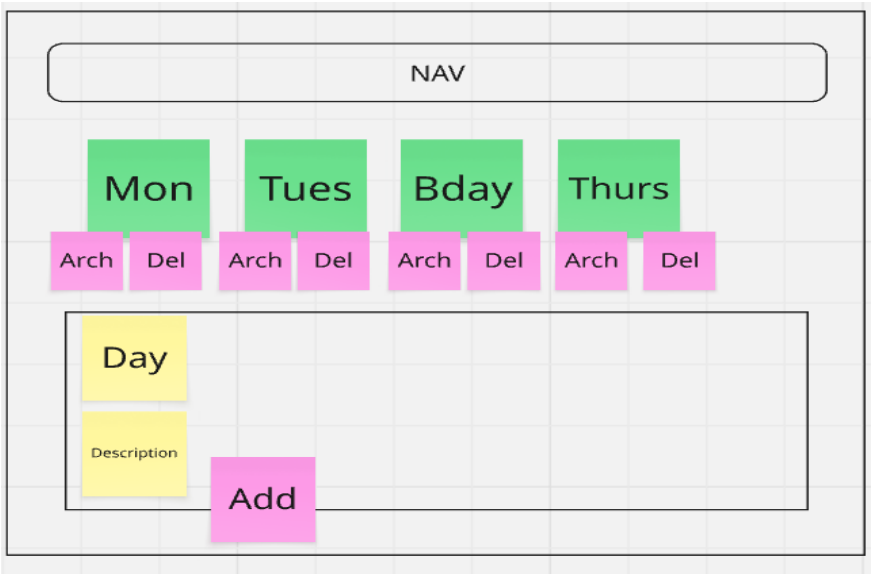


Figure 5

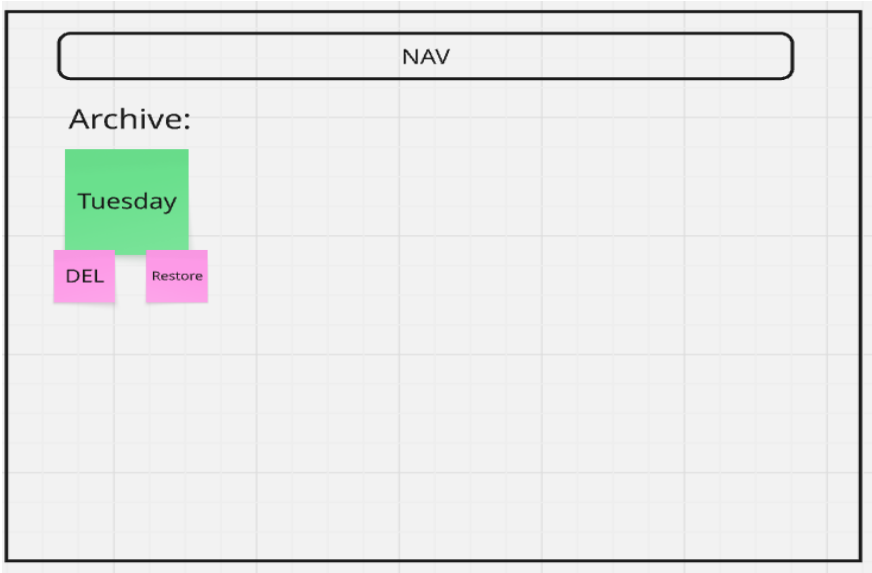


Figure 6

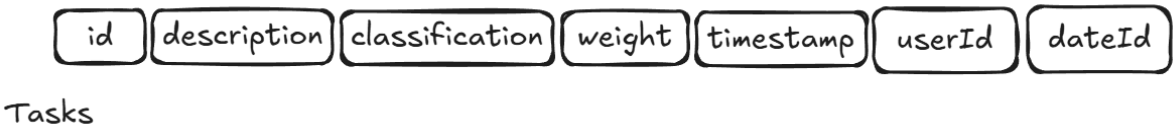


Figure 7

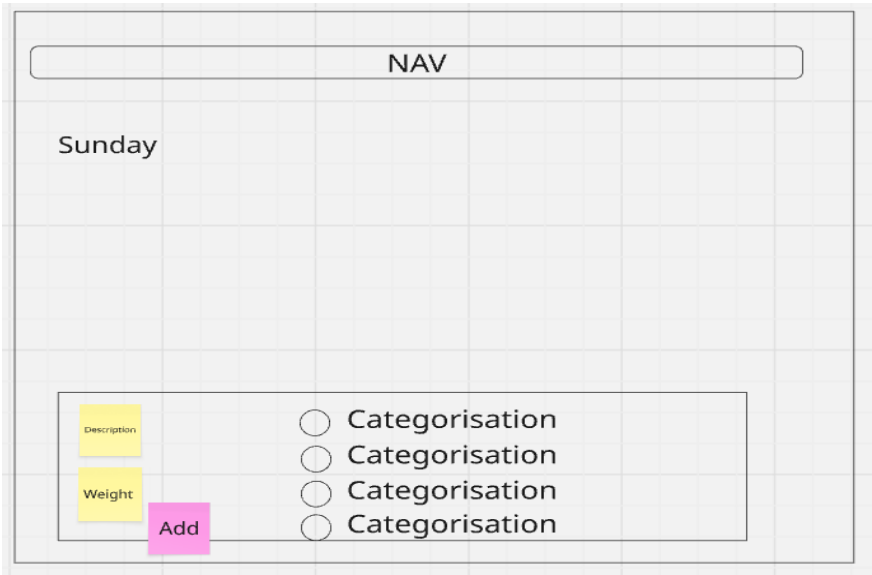


Figure 8

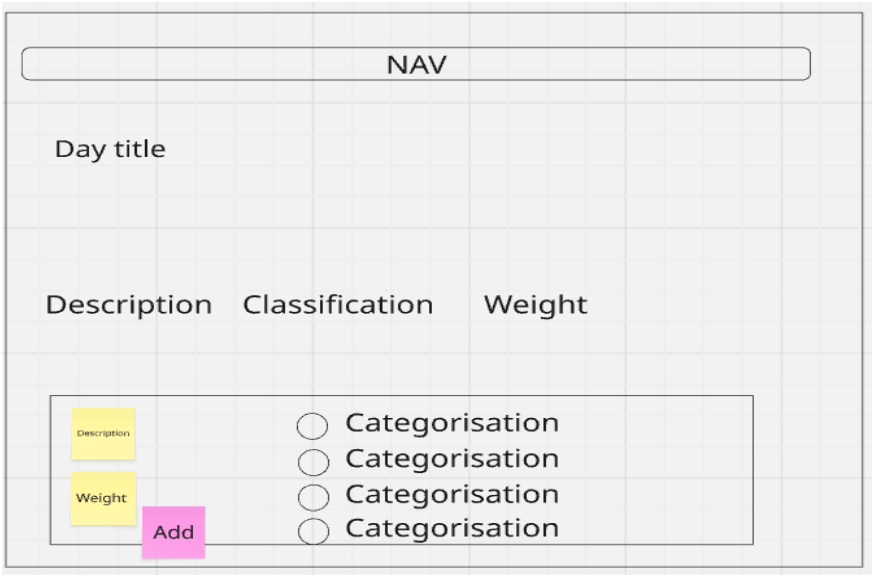


Figure 9

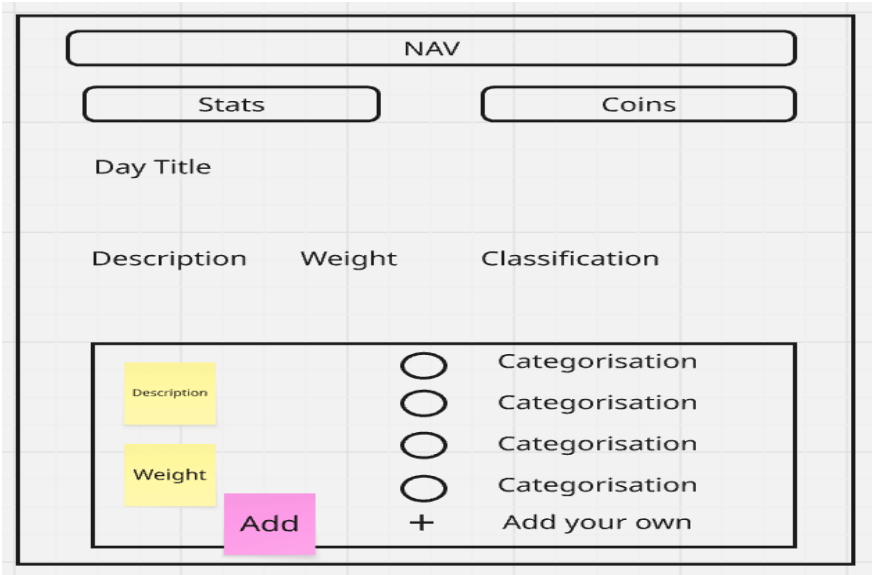


Figure 10

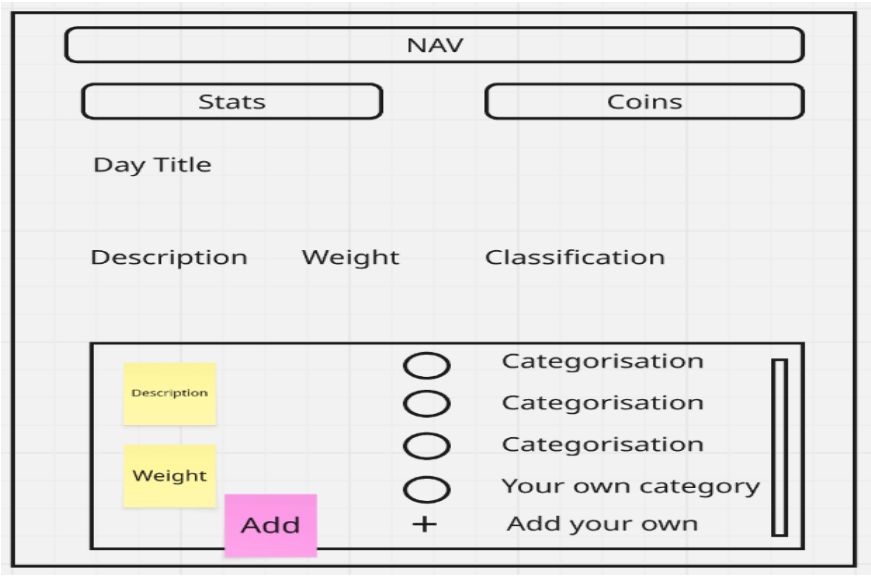


Figure 11

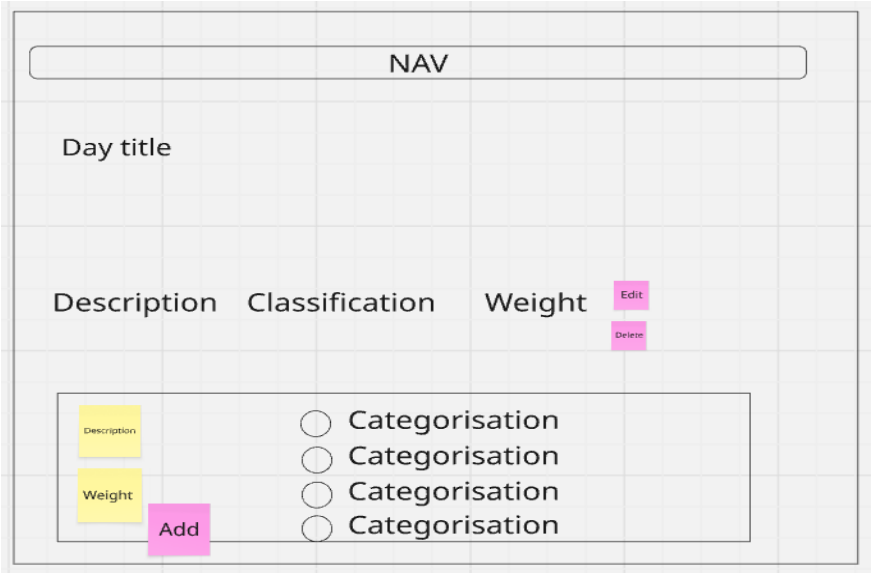


Figure 12

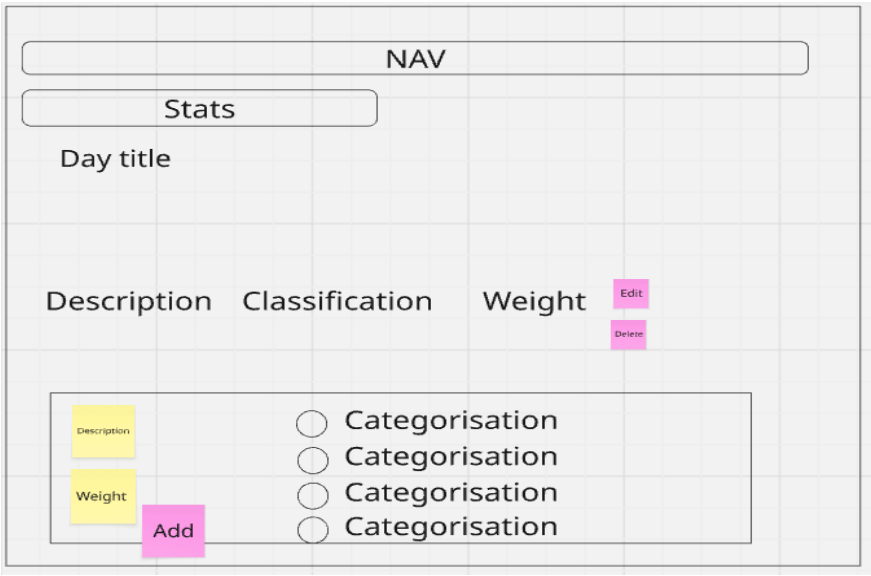


Figure 13

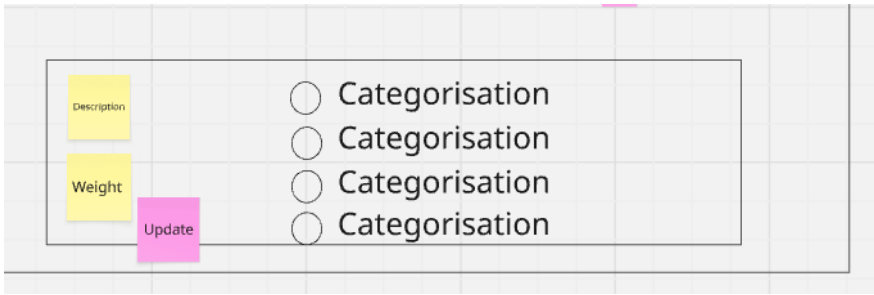


Figure 14

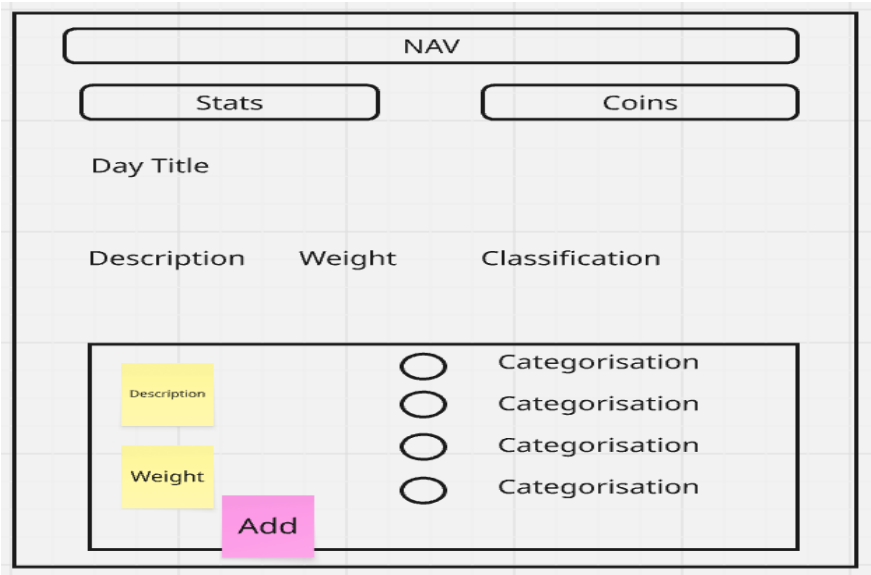


Figure 15

Figure 16

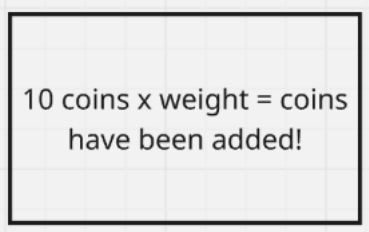


Figure 17

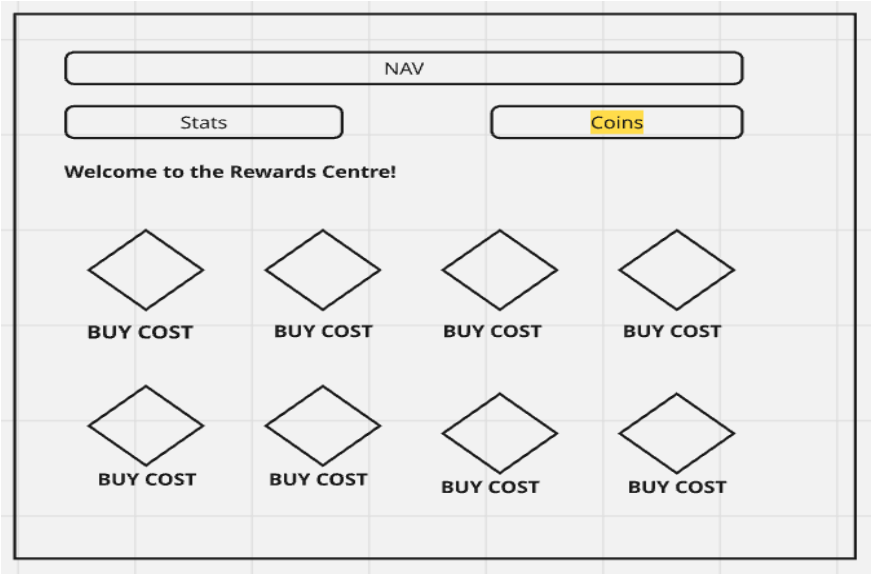


Figure 18

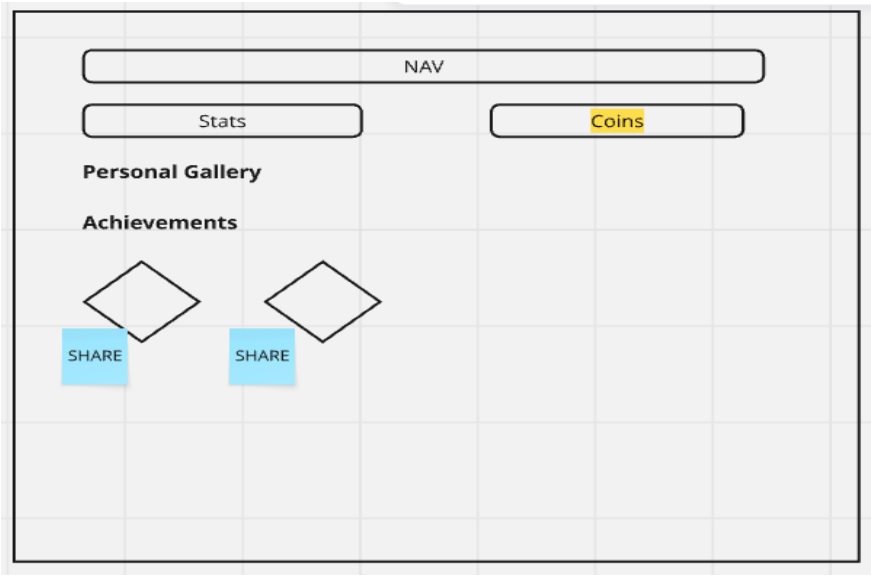


Figure 19

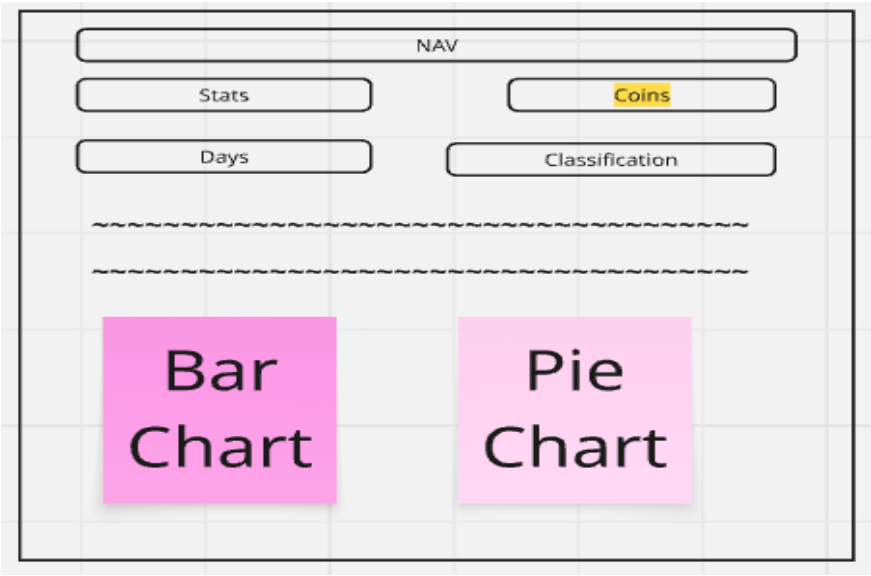


Figure 20

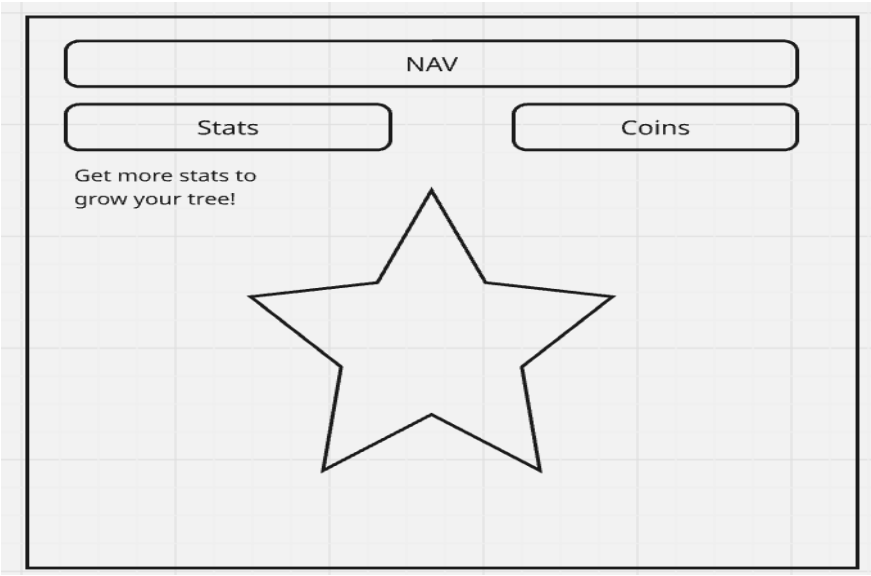


Figure 21

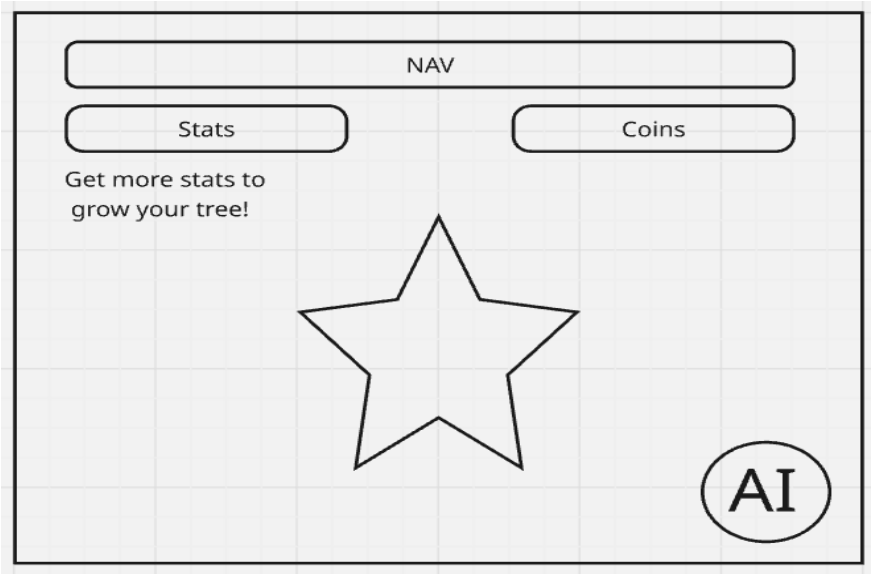


Figure 22

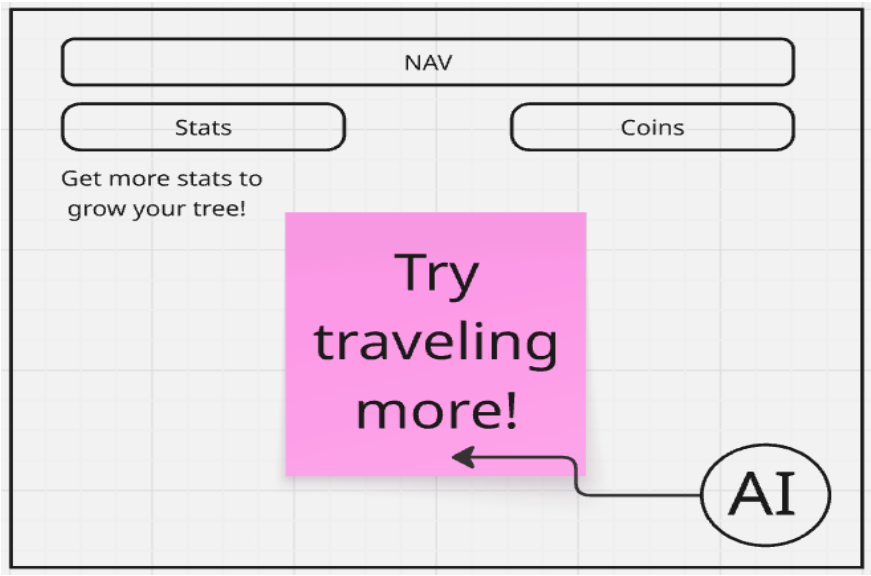


Figure 23

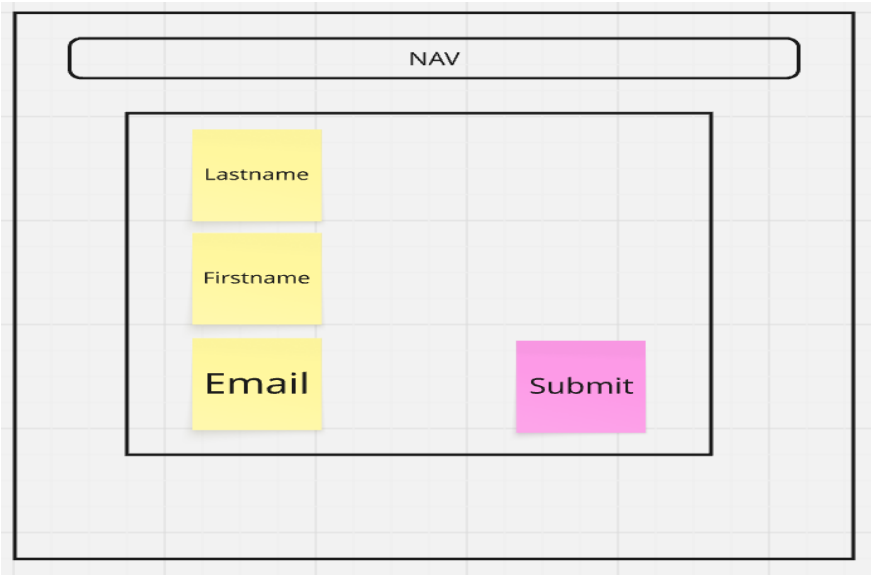


Figure 24

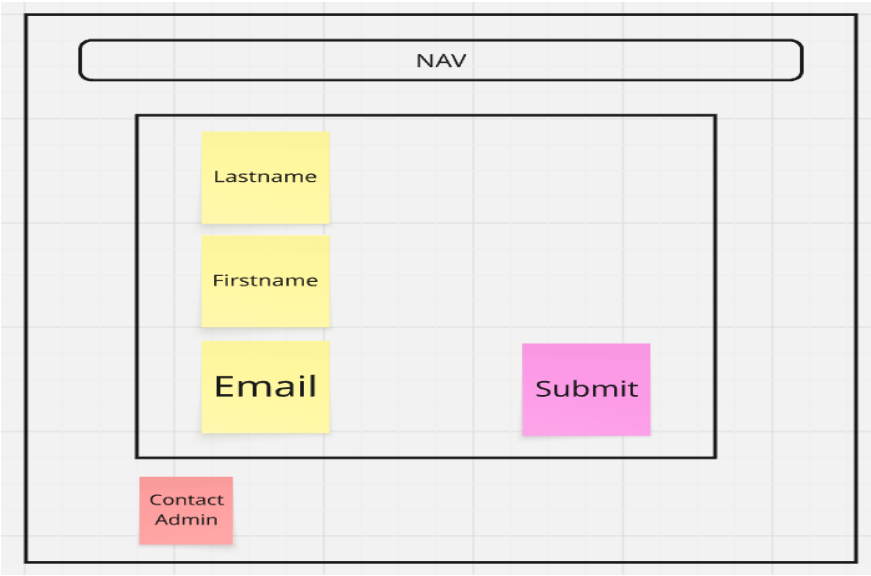


Figure 25

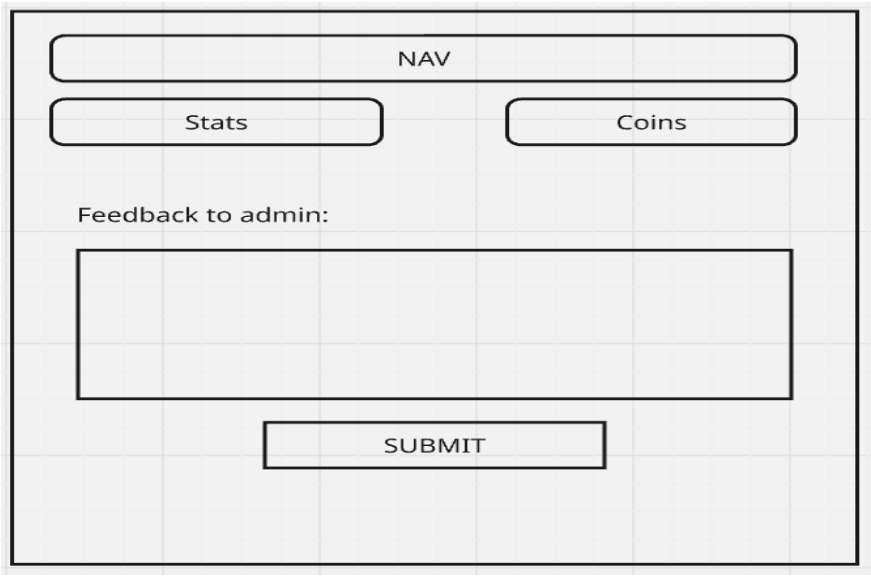


Figure 26

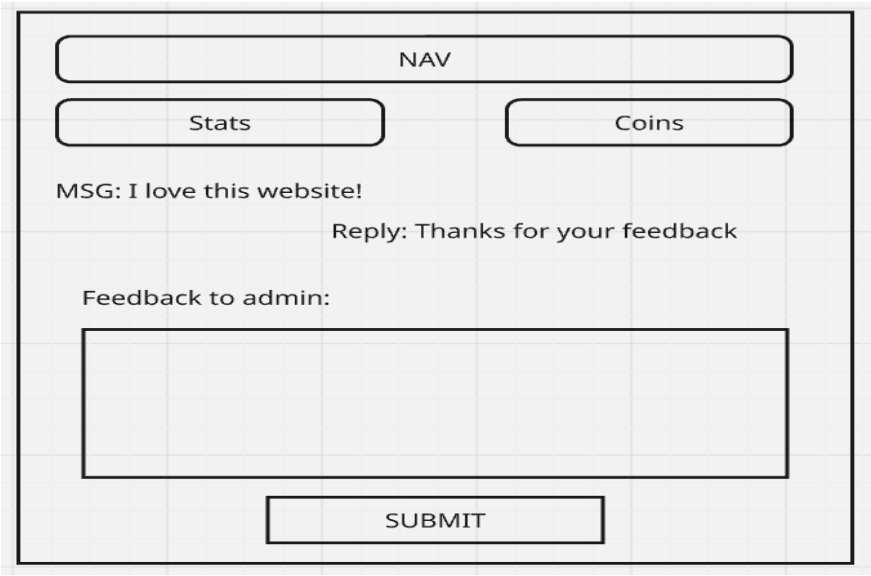


Figure 27

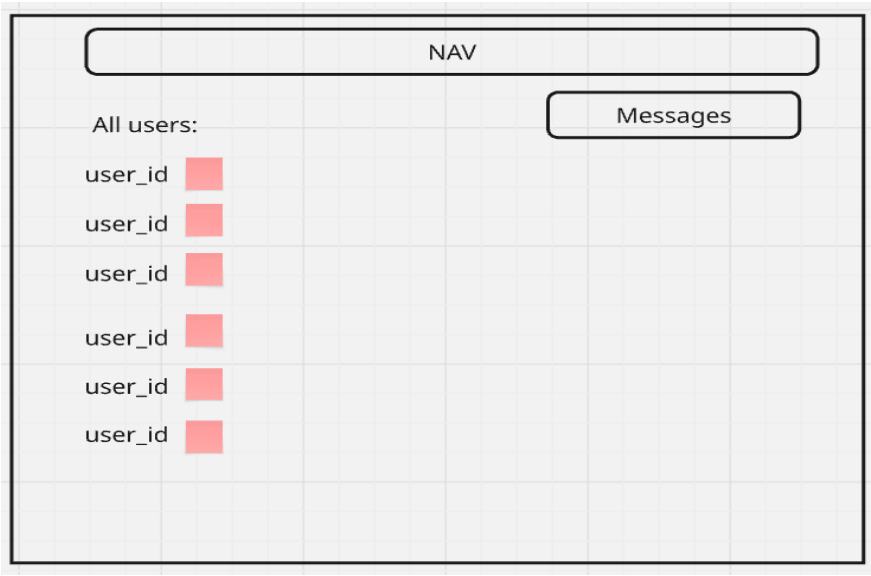


Figure 28

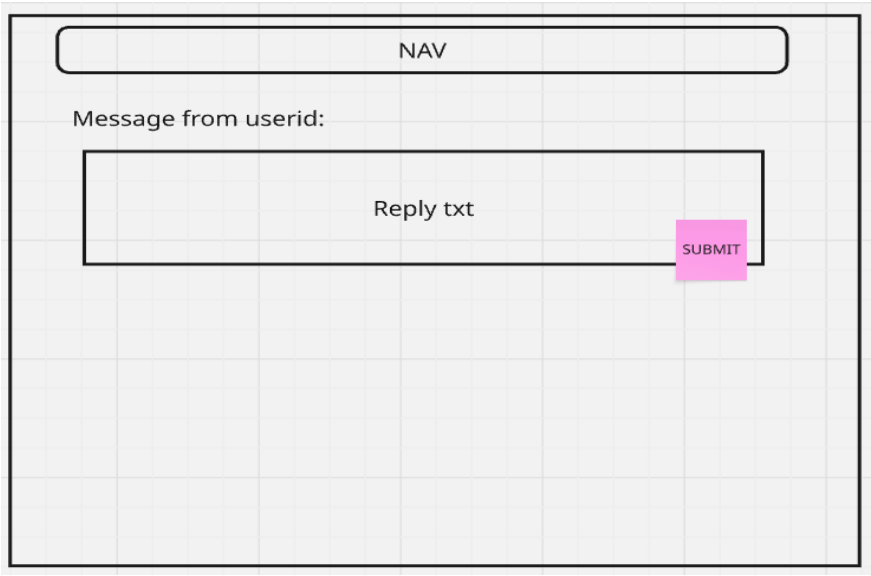


Figure 29

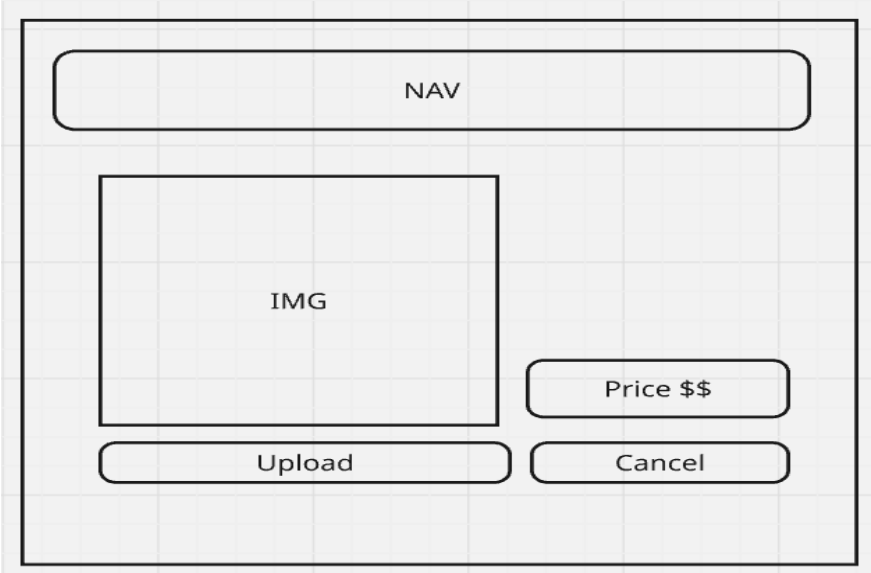


Figure 30

(excalidraw)

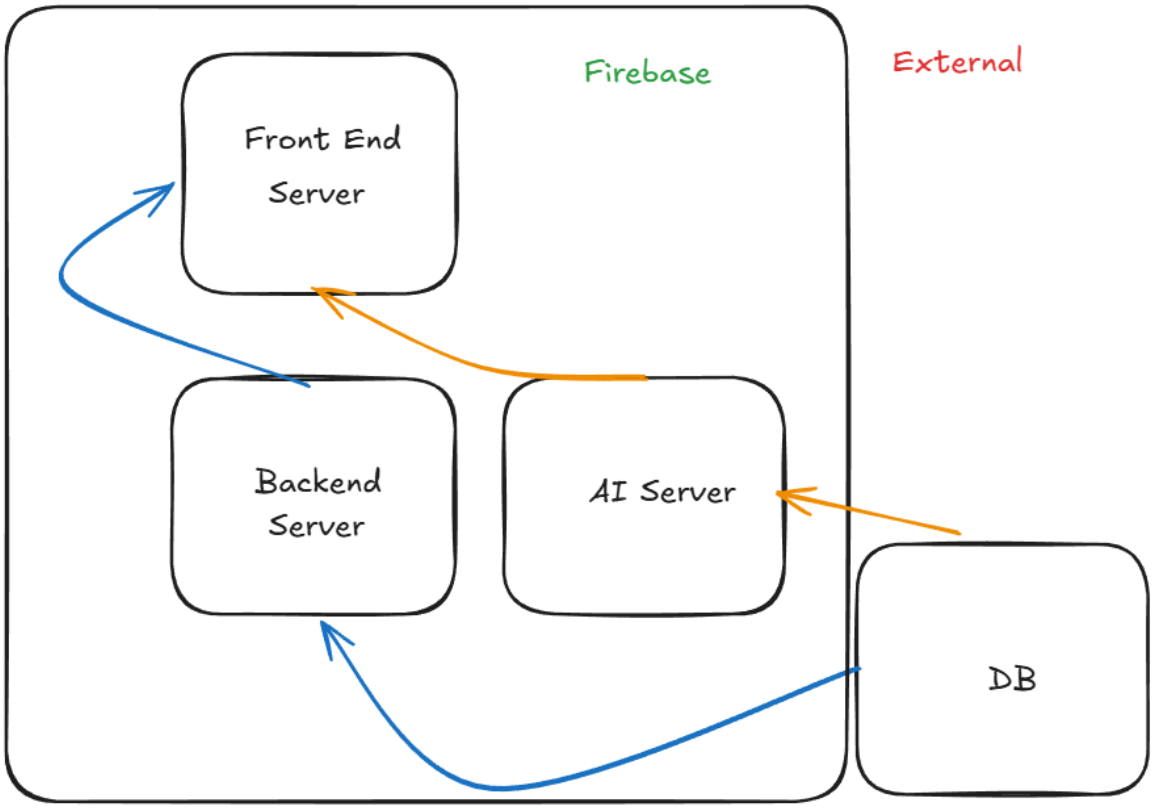


Figure 31

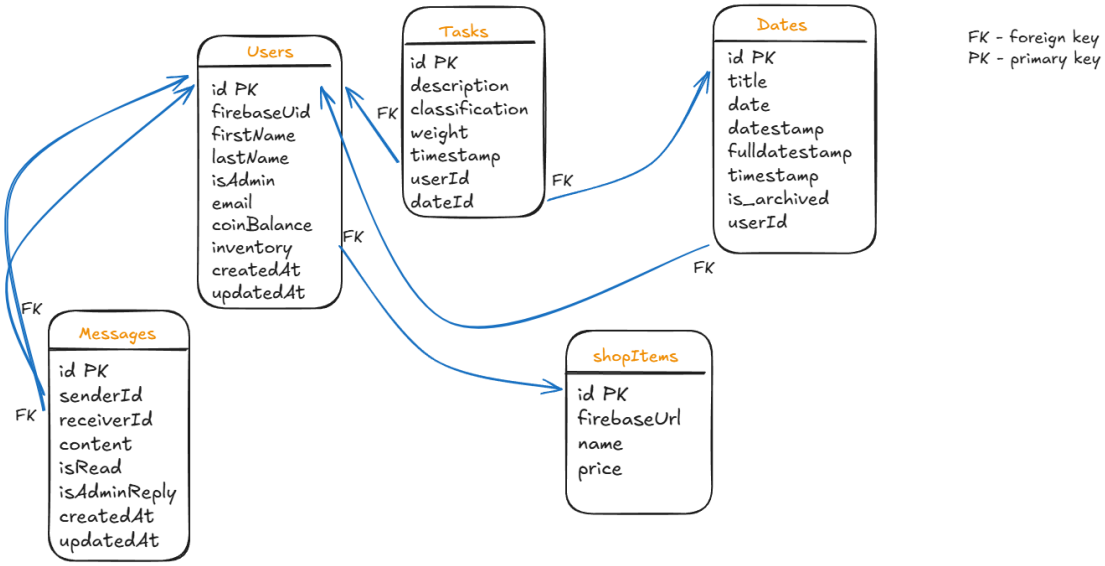


Figure 32

Bibliography:

Macori, G., et al. (2025) ‘Gamified Academic Integrity and Ethical Practices: Bridging Digital Literacy and Ethics through Interactive Scenarios’, *Proceedings of the 11th International Conference of the Immersive Learning Research Network (iLRN)*. Dublin: University College Dublin, pp. 10–18.

Jonauskaite, D. and Mohr, C. (2025) 'Do we feel colours? A systematic review of 128 years of psychological research linking colours and emotions', *Psychonomic Bulletin & Review*, 32(4), pp. 1457–1486

Weinstein, Aviv M., and Lejoyeux, Michel. 'New Developments on the Neurobiological and Pharmaco-genetic Mechanisms Underlying Internet and Videogame Addiction.' *The American Journal on Addictions* 24, no. 2 (2015): 117–125.